DISTRIBUTED COMPUTING & BIG DATA ANALYTICS

Complete Study Guide — TU Dresden M.Sc. Computer Science

From Fundamentals to Research Level

PART I: FUNDAMENTALS OF DISTRIBUTED COMPUTING

1. Introduction to Distributed Computing

1.1 What is Distributed Computing?

Definition: A computer program that runs within a distributed system is called a distributed program, and distributed programming is the process of writing such programs.

Core Concept: Distributing workload = Parallel processing (aka "concurrent computing", "parallel computing", "distributed computing")

Key Characteristics:

- Multiple autonomous computers that appear to the user as a single coherent system
- Same interface everywhere (transparency)
- Message passing for communication (e.g., MPI in HPC context)
- Middleware layer abstracts local OS differences

1.2 Why Distributed Computing?

Driving Forces:

- Processing of large amounts of data only possible due to hardware evolution
- New concepts concerning architecture, high performance computing, and processing
- Need to handle data that doesn't fit on single machines
- Fault tolerance through redundancy
- Geographical distribution of users and data

1.3 When to Use Distributed Computing?

- Data volume exceeds single machine capacity
- Computation requires parallel processing
- Need for fault tolerance and high availability
- Geographic distribution of users/data
- Cost efficiency through commodity hardware

1.4 Evolution of Distributed Systems (1955–2018)

Era	Technology	Characteristics
1955	Mainframe	Centralized, expensive

Table 1 – continued

Era	Technology	Characteristics
1962	Cluster	Packet switching, early networking
1967	Network Computing	ARPANET, datagram
1978	Home Computer	TCP/IP, UDP, Unix
1994	WWW	HTTP, HTML
1999	Grid Computing, P2P	Middleware, distributed collaboration
2004	Mobile Computing	Wireless, portable
2006	Cloud Computing	Virtualization, hypervisors, easy scaling
2008	IoT	Connected devices, sensors
2009	Fog/Edge Computing	Infrastructure at the edge, reduced latency
2018	Continuum Systems	Seamless integration, edge-to-cloud

Centralization Decentralization Spectrum:

- Mainframe (highly centralized) → Cloud (centralized) → Edge/Fog (decentralized)

1.5 Key Challenges in Distributed Computing

Process/Analysis Challenges:

- Scalability: Handle massive amounts of data
- Data management: Storage, retrieval, organization
- (Data-)security and quality: Integrity, confidentiality

Management of Infrastructure:

- (Heterogeneous) instances: Different hardware/software
- Service orientation: Modular, reusable services
- Fault tolerance: Handle failures gracefully
- Communication: Efficient data exchange
- Security: Authentication, authorization, encryption

---

2. Big Data Fundamentals

2.1 The 5 Vs of Big Data

V	Description	Key Points
Volume	Data at Rest	Terabytes to exabytes of existing data to process
Velocity	Data in Motion	Streaming data, milliseconds to seconds to respond
Variety	Data in Many Forms	Structured, unstructured, text, multimedia
Veracity	Data in Doubt	Uncertainty due to inconsistency, incompleteness, ambiguities

Table 2 – continued

V	Description	Key Points
Value	Extract new content	The ultimate goal: derive meaningful insights

Additional dimensions: Privacy, Human Interaction

2.2 Data Scale Reference

Scale	Size	Real-world Example
Megabyte (10)	~1 MB	Small documents
Gigabyte (10)	~1 GB	Movie file
Terabyte (10^{12})	~1 TB	Facebook: 500 TB per day
Petabyte (10^{1})	~1 PB	CERN: 1 PB/second
Exabyte (10^{1})	~1 EB	Global internet traffic
Zettabyte (10^{21})	~1 ZB	Network traffic 2016
Yottabyte (10^{2})	~1 YB	250 trillion DVDs
Brontobyte (10^{2})	~1 BB	Future sensor data

2.3 Sources of Big Data

- **Sensor Data:** IoT devices, industrial sensors, weather stations
- **Mobile Revolution:** Smartphones, GPS, mobile apps
- **Event Analysis:** User behavior, click streams, transactions
- **Social Media:** Facebook, Twitter, Instagram
- **Scientific Instruments:** Telescopes, microscopes, gene sequencers
- **Enterprise Systems:** ERP, CRM, logs

2.4 Why Collect and Analyze Huge Data?

Basic Goals:

- Derive knowledge out of data/information
- Use analytics to find correlations/causality, systematic behavior
- Use machine learning or AI approaches

Business Drivers:

- Improving customer experience: Optimization, direct feedback, recommendations
- Refining marketing strategy: Quantify response, “understand” your customer
- Turning data into cash-flow: Secondary data markets, behavioral data sharing
- Using data to secure data: Two-factor authorization (voice, biometric data)

2.5 Scientific Computing vs. Big Data Analytics

Aspect	Scientific Computing (HPC)	Big Data Analytics
Approach	Theory-based hypothesis	Data-driven approaches
Example	Relativistic theory → gravitational waves	No theory available for socio-economic systems
Methods	Mathematical models, simulations	Statistics, ML, pattern discovery
Data	Often simulated/synthetic	Real-world, messy, heterogeneous
New Trends	Physics-informed networks, Digital Twins	Fusion of simulated and measurement data

---

3. Data Processing Pipeline

3.1 The Pipeline Stages

```
Data Collection → Extraction/Cleaning/Annotation → Integration/Aggregation → Analysis/Modelling →
↪  Interpretation
```

Underlying Factors:

- Volume, Veracity, Velocity, Variety, …
- Privacy considerations
- Human Interaction
- → **Value extraction**

3.2 Data Lifecycle

```
Data created/collected → Data processed → Data analyzed → Data published → Data archived → Data re-used
↪  → (back to start)
```

3.3 Key Challenges in Data Processing

1. **Feature extraction:** Need for meaningful features from raw data
2. **Data portability:** Analysis may require specific data types
3. **Data cleaning:** Handle erroneous, inconsistent, or missing data
4. **Data integration:** Data from multiple sources
5. **Data reduction and transformation:** Efficiency of analysis

3.4 Data Preprocessing Techniques

- **Data Cleaning:** Remove noise, correct inconsistencies
- **Data Normalization:** Scale data to common range
- **Data Transformation:** Convert formats, discretization
- **Missing Values Imputation:** Fill gaps (mean, median, ML-based)
- **Data Integration:** Merge from multiple sources
- **Noise Identification:** Detect and handle outliers

PART II: SCALING PARADIGMS

4. Vertical vs. Horizontal Scaling

4.1 Vertical Scaling (Scale-Up)

Definition: Use more computing elements in a single system/application

Characteristics:

- Usually within a single system
- Add more CPUs, memory, storage to existing machine
- Often communication intensive (iterative approaches)
- Ideal case: Highly parallel simulations

Pros:

- Simpler management (single machine)
- Better for tightly-coupled applications
- Lower latency for inter-process communication

Cons:

- Hardware cost increases exponentially
- Single point of failure
- Physical limits to expansion
- Vendor lock-in

4.2 Horizontal Scaling (Scale-Out)

Definition: Add further instances of the same kind

Characteristics:

- Many machines (hundreds, thousands)
- Opposite to scale-up
- No special assumptions concerning hardware dependencies
- Ideal case: Distribute computing to data, not vice versa
- "Separate" partitioned workloads

Pros:

- Cost-effective (commodity hardware)
- Fault tolerant (no reliance on single instance)
- Virtually unlimited scalability
- Flexible resource allocation

Cons:

- Complex distributed coordination

- Network communication overhead
- Data consistency challenges
- Requires distributed programming expertise

4.3 Comparison Table

Aspect	Vertical Scaling	Horizontal Scaling
Cost	Expensive hardware	Cheap commodity hardware
Limit	Hardware ceiling	Virtually unlimited
Fault Tolerance	Low (single point)	High (distributed)
Complexity	Lower	Higher
Use Case	Databases, monolithic apps	Big Data, web services
Examples	Mainframes, large servers	Hadoop, Spark clusters

PART III: HADOOP ECOSYSTEM (First Generation)

5. Hadoop and MapReduce

5.1 Historical Context

2000s Era Hardware:

- Disk space cheap (primary storage)
- Network was costly
- RAM very expensive
- Single-core machines dominant

Software of 2000s:

- Object orientation, optimization for single core
- SQL as primary analysis language
- Specific frameworks (MATLAB)

~2010s Era Hardware:

- RAM/Flash cheaper and faster → primary storage
- Network faster, virtualization
- Multi-core machines dominating
- Different architectures (GPUs, TPUs)

Software Evolution:

- More functional programming and frameworks
- Multicore programming and distribution
- NoSQL alternatives

5.2 First Generation (2000s): Hadoop

Background:

- Only few companies had real big data needs
- Batch processing dominant (reporting++)
- Primarily volume concern (not full 5Vs)
- Mostly used for search/basic behavior analysis (logging data)

Hadoop Implementation:

- Simple programming approach
- Batch orientated
- Underlying HDFS for data distribution
- Open-source framework cloning Google's MapReduce

5.3 MapReduce Paradigm

Origin:

- Google File System (Ghemawat et al., SOSP 2003)
- MapReduce: Simplified Data Processing on Large Clusters (Dean & Ghemawat, OSDI 2004)

Core Idea:

```
Input → Split → Map → Shuffle/Sort → Reduce → Output
```

Map Phase:

- Processes input data in parallel
- Extracts relevant data, emits (key, value) pairs
- "Pure" parallel stage

Shuffle Phase:

- Synchronization of data
- Groups all values by key
- Network-intensive operation

Reduce Phase:

- Aggregates values for each key
- Produces final output
- "Pure" parallel stage

5.4 Word Count Example (Classic MapReduce)

```
Input: "Deer Bear River Car Car River Deer Car Bear"

Splitting:
```

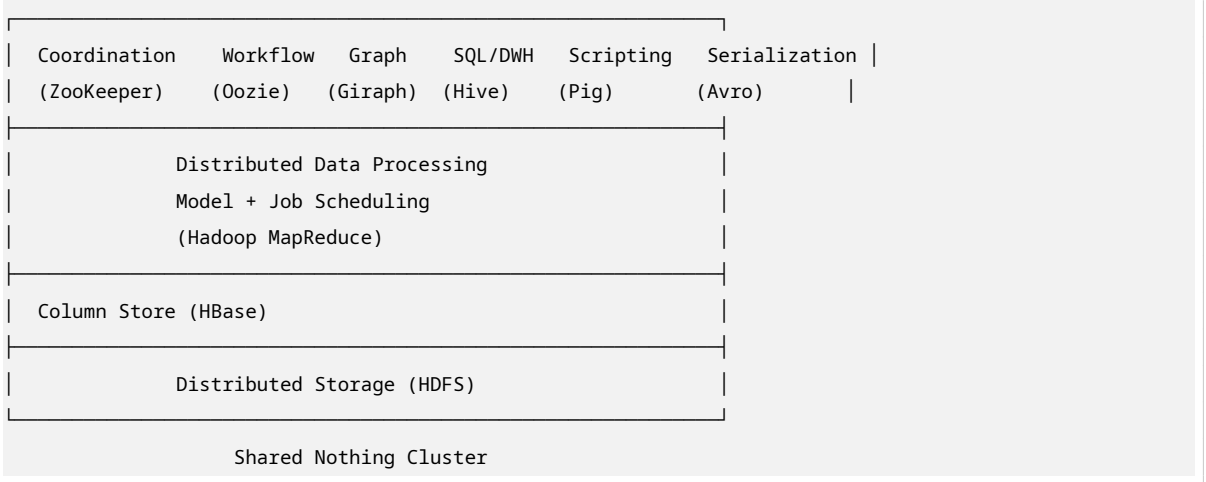
```
  Split 1: "Deer Bear River"
  Split 2: "Car Car River"
  Split 3: "Deer Car Bear"

Mapping:
  Split 1 → (Deer,1), (Bear,1), (River,1)
  Split 2 → (Car,1), (Car,1), (River,1)
  Split 3 → (Deer,1), (Car,1), (Bear,1)

Shuffling:
  Bear: [1, 1]
  Car: [1, 1, 1]
  Deer: [1, 1]
  River: [1, 1]

Reducing:
  Bear: 2
  Car: 3
  Deer: 2
  River: 2
```

5.5 Hadoop 1.0 Ecosystem Architecture



5.6 HDFS (Hadoop Distributed File System)

Characteristics:

- Open Source under Apache License 2.0
- Initially created at Yahoo!
- Written entirely in Java
- Typically runs on GNU/Linux

Architecture:

- **NameNode (Master):** Manages metadata, file system namespace
- **DataNodes (Workers):** Store actual data blocks

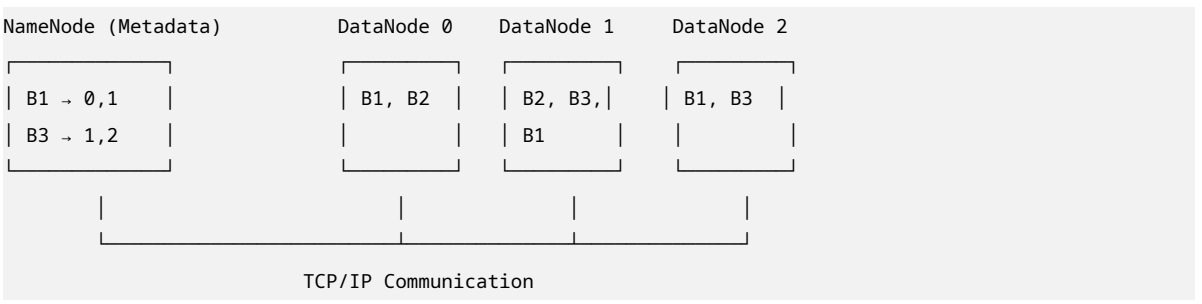
- **Blocks:** Files broken into block-sized chunks (default 128MB)

Key Design Decisions:

- **Hardware Failure:** Hundreds to thousands of commodity servers; failure is the norm
- **Streaming Data Access:** Batch processing rather than interactive use
- **Large Data Sets:** Tens of millions of files, GB to TB each
- **Moving Computation to Data:** Computation "near" the data it operates on
- **Portability:** Run on heterogeneous hardware and software platforms

Block Replication:

- No RAID, but real duplicates
- Default replication factor: 3
- Blocks distributed across DataNodes for fault tolerance



5.7 MapReduce Criticism (DBMS Perspective, 2008)

Criticism: Programming paradigm is a step backwards for large-scale data intensive applications

Missing Core DBMS Features:

1. **Indexing:** Simple MapReduce has no index, only "brute force" scans
2. **Updates:** Cannot change data in the database easily
3. **Transactions:** No parallel update & recovery from failures during update
4. **Integrity constraints and references:** No referential integrity
5. **Views:** Schema changes require rewriting application programs

5.8 MapReduce Limitations

Fundamental Limitations:

- **Non-Interactive:** Batch processing, high latency
- **No Iterations:** Cannot easily express iterative algorithms (ML, graph processing)
- **Simple but Complex:** Simple programming model but complex to transform tasks
- **Disk I/O Bottleneck:** Every MapReduce job writes to disk

Processing Model Limitations:

1. Map Only: Simple filtering, no aggregation

2. Classic MapReduce: Single pass, no iteration
3. Iterative MapReduce: Requires external loops, inefficient
4. Point-to-Point: Graph processing difficult
5. Map-Streaming: Event processing limitations
6. Shared Memory: Not natively supported

---

6. Hadoop Ecosystem Extensions

6.1 Apache Hive

Definition: Data warehouse for MapReduce

Key Features:

- Strongly influenced by data warehouse concepts
- Hive = MapReduce + SQL
- SQL simple to use, keeps MapReduce scalability and fault tolerance
- HiveQL = SQL-like query language
- No need to implement queries in low-level Java API
- Extensible with MapReduce scripts

Architecture:

```
User → Hive SQL → [CLI/JDBC/ODBC/Web UI] → HiveServer2 → Compiler/Optimizer/Executor
                                                                    ↓
                                                          Map/Reduce → Hadoop (NameNode, JobTracker,
                                                          ↪  TaskTracker, DataNode)
```

Query Flow:

1. User issues SQL query
2. Hive parses and plans query
3. Query converted to MapReduce
4. MapReduce tasks run by Hadoop

Usage:

- First released 2010, developed by Facebook
- Used by Netflix, FINRA (Financial Industry Regulatory Authority)
- Amazon Elastic MapReduce includes Hive

Example HiveQL:

```
DROP TABLE IF EXISTS docs;
CREATE TABLE docs (line STRING);
LOAD DATA INPATH 'input_file' OVERWRITE INTO TABLE docs;
```

```
CREATE TABLE word_counts AS
SELECT word, count(1) AS count
FROM (SELECT explode(split(line, '\s')) AS word FROM docs) temp
GROUP BY word
ORDER BY word;
```

6.2 Other Hadoop Ecosystem Components

- **HBase:** Column-oriented NoSQL database
- **Pig:** High-level scripting platform
- **Giraph:** Graph processing
- **Oozie:** Workflow scheduling
- **ZooKeeper:** Coordination service
- **Avro:** Data serialization

PART IV: SECOND GENERATION — IN-MEMORY & STREAMING

7. In-Memory Processing

7.1 Why In-Memory?

Previously:

- Processing based on disk storage and relational databases
- Using SQL query language, but became too slow with increasing data

Now:

- Processing of data stored in in-memory (database)
- Towards data analysis in real time
- Lifts limitations of IO-waiting times (usually show stopper in IO-intense applications)
- RAM has come down in cost
- Provides basis for other algorithm pipelines: Iterations, Joins

History:

- Triggered by business intelligence (BI) needs
- “Coffee break analytics”: Start analytic report, results come later

7.2 In-Memory Hardware Techniques

Technique	Description	Examples	Limitations
Caching	Store frequently accessed data copies	Processor caches, file cache, disk cache	Works only if working set is small
Replication	Duplicate data across nodes	RAID, CDN, Web/ Server Caches	Expensive, limited by receiving side

Table 6 – continued

Technique	Description	Examples	Limitations
Prefetching	Predict and pre-fetch needed data	Disk caches, Google Earth	Only works with predictable access patterns

Application behavior efficiency depends on actual workflow.

---

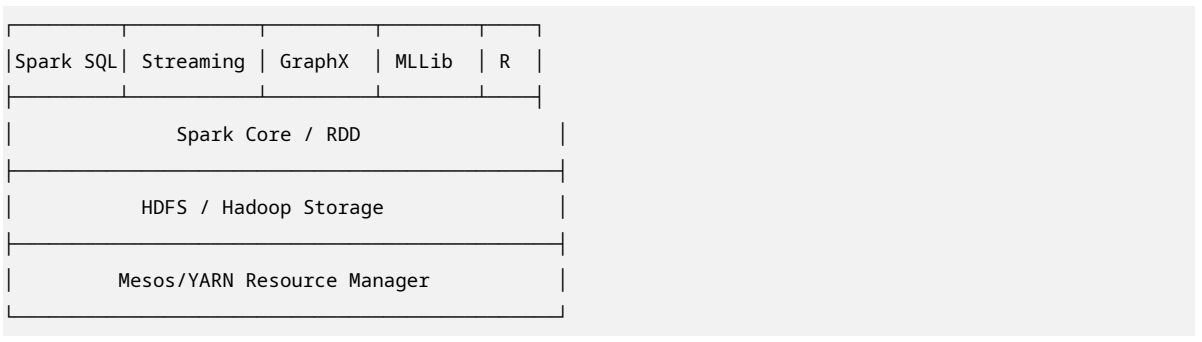
8. Apache Spark

8.1 Introduction

- **Created:** AMPLab (Berkeley, 2009), now Databricks
- **Written in:** Scala
- **License:** Apache Foundation
- **Community:** Strong international developer community

Definition: A fast and general engine for large-scale data processing

8.2 Spark Architecture Stack



8.3 Spark Core Features

- **DAG Execution Engine:** Directed Acyclic Graph execution engine
- **Cyclic Data Flow:** Supports complex workflows
- **In-Memory Computing:** Keeps data in RAM between operations
- **80+ High-Level Operators:** For building parallel applications
- **Interactive Shells:** Scala, Python, R shells
- **Unified Stack:** Combine SQL, streaming, ML, graph in same application

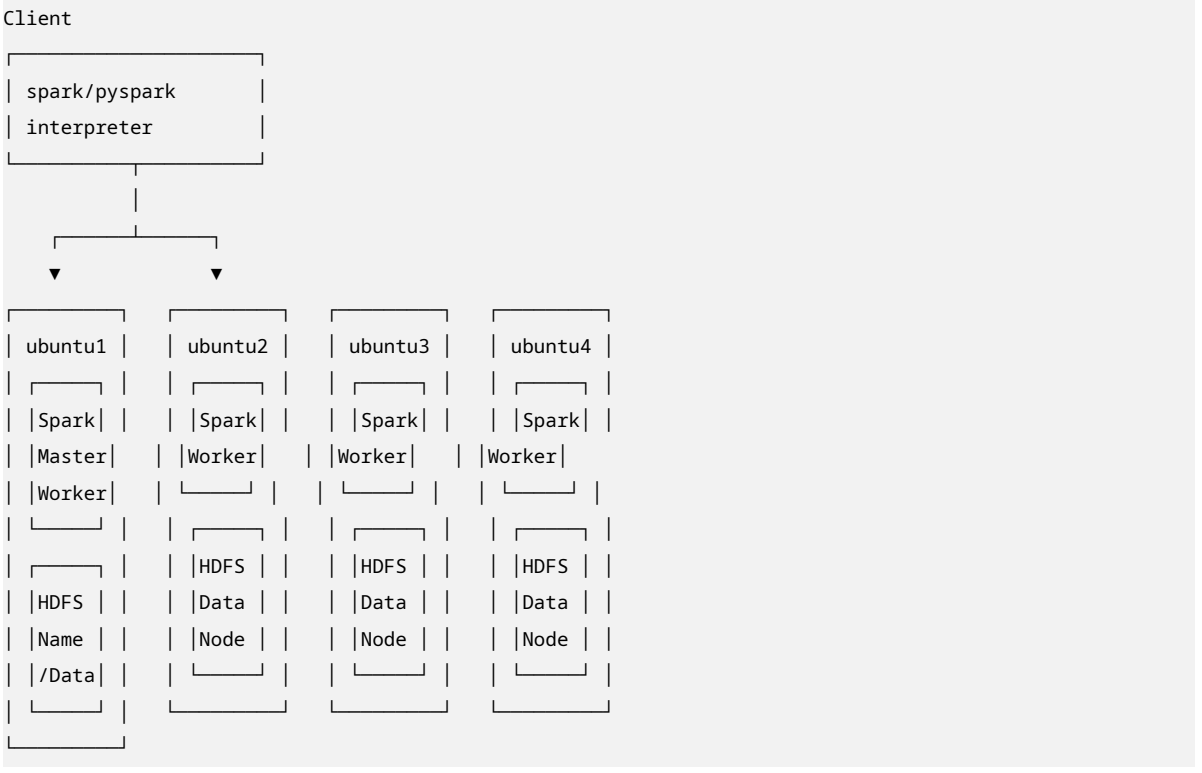
8.4 Spark vs. Hadoop Performance

- **Logistic Regression Example:**
  - Hadoop: 110 seconds
  - Spark: 0.9 seconds
  - ~**120x faster!**
- **Iterative Algorithms:**
  - Hadoop: Linear increase with iterations (reads from disk each time)
  - Spark: Near-constant time (keeps data in memory)

8.5 Spark Deployment Modes

- Standalone cluster
- Hadoop YARN
- Apache Mesos
- Amazon EC2
- Kubernetes (modern)

8.6 Spark Cluster Architecture



8.7 Spark Context and Execution

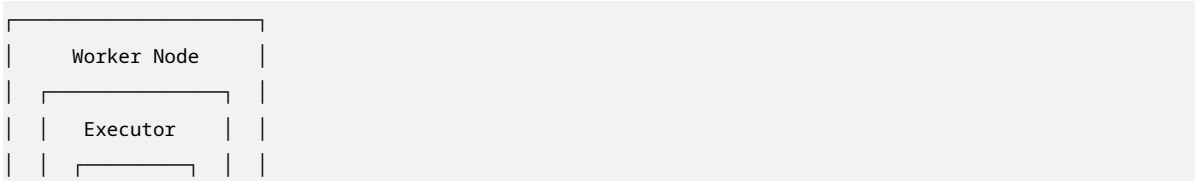
SparkContext: Connection of client to available cluster

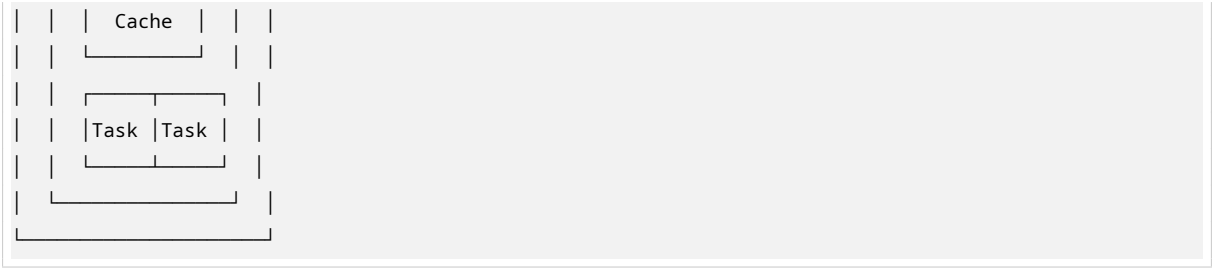
- Master defines cluster connection: local / Spark standalone / Mesos / EC2

Execution Flow:

1. Connects to cluster manager which allocates resources
2. Acquires executors on worker nodes
3. Sends driver application code to executors
4. Distributes tasks for executors to run separately

Worker Node Structure:





8.8 Spark Application Isolation

- Each application has its own private driver and executors
- Applications cannot access each other's data (isolation)
- Multiple applications can run on same cluster

---

9. Resilient Distributed Datasets (RDDs)

9.1 What is an RDD?

Definition: Resilient Distributed Dataset — Spark's core data structure

Properties:

- **Resilient:** Fault-tolerant with help of RDD lineage graph
- **Distributed:** Data residing on multiple nodes in cluster
- **Dataset:** Collection of partitioned data with primitive values or objects (tuples, etc.)

9.2 RDD Operations

Two Types of Operations:

Type	Description	Examples
Transformations	Lazy operations that return another RDD	map, filter, flatMap, groupByKey, reduceByKey
Actions	Operations that trigger computation and return values	collect, count, reduce, saveAsTextFile

Lazy Evaluation:

- Transformations build up a DAG of operations
- Nothing executes until an action is called
- Optimizer can rearrange operations for efficiency

9.3 RDD Lineage and Fault Tolerance

```
RDD (immutable)
     |
     ▼ pointer to parent
Transformations (map, filter, ...)
     |
     ▼
New RDD
     |
     ▼
Actions (saveAsTextFile, reduce, ...)
     |
     ▼
Save/Display
```

Fault Recovery: If a partition is lost, recompute from lineage (parent RDDs)

9.4 RDD Example Code

```
// Create RDD from HDFS
val lines = spark.textFile("hdfs://...")

// Transformation: filter
val errors = lines.filter(_.startsWith("ERROR"))

// Chain of transformations + action
errors.filter(_.contains("HDFS"))
      .map(_.split('\t')(3))
      .collect()  // Action triggers execution
```

DAG Execution:

```
lines —filter()—→ errors —filter()—→ HDFS errors —map()—→ time fields —collect()—→ output
```

9.5 Parallelism Control

- Controlled by RDD using 'repartition' method
- Spark tries to distribute computing to already partitioned data
- Avoids data exchange when possible
- Data locality optimization

---

10. Data Structure Evolution in Spark

10.1 RDD (2011)

- Distributed collection of JVM objects
- Functional operators (map, filter, etc.)
- Low-level, flexible, but no optimization

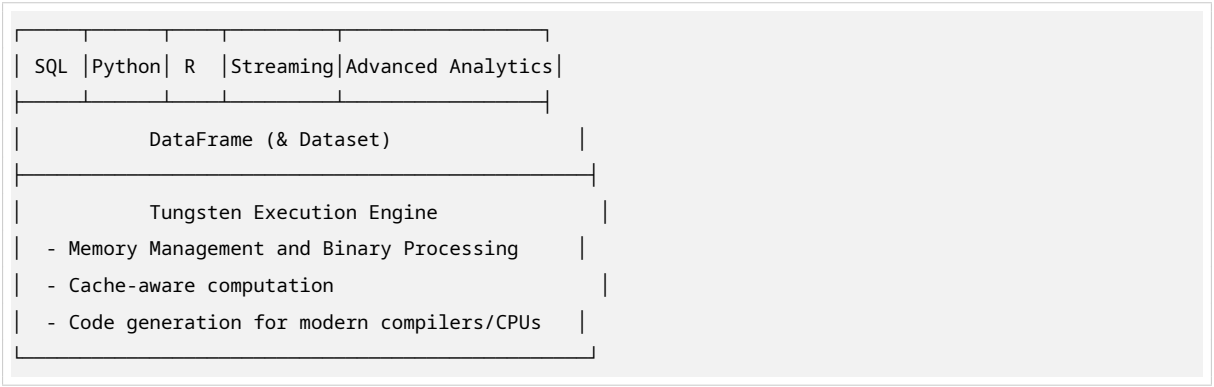
10.2 DataFrame (2013)

- Distributed collection of Row objects
- Expression-based operations and UDFs (User Defined Functions)
- Logical plans and optimizer (Catalyst optimizer)
- Fast/efficient internal representations
- Schema-aware

10.3 Dataset (2015)

- Internally rows, externally JVM objects
- "Best of both worlds": type safe + fast
- Combines RDD type safety with DataFrame optimization

10.4 Enhanced Stack (Modern Spark)



PART V: STREAM PROCESSING

11. Stream Processing Fundamentals

11.1 Batch vs. Stream Processing

Aspect	Batch Processing	Stream Processing
Data	Fixed, bounded	Continuous, unbounded
Query	One-time query	Continuous (standing) query
Latency	Minutes to hours	Milliseconds to seconds
Throughput	High	Moderate to high
Use case	Historical analysis	Real-time monitoring

Batch Model:

```
Query → DB → Results
```

Stream Model:

```
Events → Queries/State → Events

       State (limited resources)
```

11.2 Why Stream Processing?

Driving Applications:

- Autonomous driving (real-time sensor fusion)
- Sensor data analysis (IoT, industrial)
- Fraud detection (financial transactions)
- Traffic monitoring (toll collection, speed control)
- Social media sentiment analysis
- Cybersecurity (intrusion detection)

11.3 Big Data Streaming Sources

- Traffic monitoring (toll stations, smartphones, GPS)
- Weather sensors (buoys, satellites)
- Industrial robots and manufacturing
- Wearable devices (fitness trackers)
- Smart home devices (smoke detectors)
- Radio telescopes and scientific instruments

---

12. Stream Processing Frameworks

12.1 Comparison of Major Frameworks

Framework	Type	Latency	Throughput	API Level
Apache Storm	True streaming	Very low	Lower	Low (Bolts, Spouts) + Trident
Spark Streaming	Micro-batching	Higher	High	Functional (DStreams)
Apache Samza	True streaming (Kafka-based)	Low	Moderate	Low level
Apache Flink	True streaming	Adjustable	High	Rich functional API

12.2 Apache Flink

Overview:

- Large-scale data processing engine (similar to Spark)
- Derived from Stratosphere project
- Many similarities to Apache Spark
- Support for iterations (ML), graph, batch processing
- **Good support for streaming**

Key Features:

- True streaming with adjustable latency-throughput trade-off
- Rich functional API exploiting streaming runtime
- Flexible windowing semantics
- Exactly-once processing guarantees with (small) state
- Data stream outputs
- Streaming of results between operators

Flink Architecture:

```
Applications & Languages: Hive, Cascading, Giraph, Mahout, Pig, Crunch
                    ↓
Data Processing Engines: MapReduce, Spark, Storm, Tez, Flink
                    ↓
App & Resource Management: YARN, Mesos
                    ↓
Storage, Streams: HDFS, HBase, Kafka
```

Flink Streaming Example (Stock Prices):

```
Stock Sources → Merge → Stock Stream → Window (10 sec, every 5 sec) → [MinBy/MaxBy/Mean Price]
                                                                  ↓
                                                          Group by symbol
```

12.3 Spark Streaming

Model: Micro-batching (not true streaming)

Architecture:

```
Input Data Stream → Spark Streaming → Batches of Input Data → Spark Engine → Batches of Processed Data
```

Input Sources: Kafka, Flume, HDFS/S3, Kinesis, Twitter **Output:** HDFS, Databases, Dashboards

Window Operations:

- Original DStream → Windowed DStream
- Window-based operations over time

12.4 Spark Streaming vs. Flink Streaming

Aspect	Spark Streaming	Flink
Processing Model	Micro-batching	True streaming
Latency	Higher (batch overhead)	Lower (event-by-event)
Throughput	High	High
State Management	Basic	Advanced
Windowing	Time-based	Flexible (time, count, session)
Exactly-once	Supported	Supported
API	DStreams/Structured Streaming	DataStream API

Visual Difference:

```
Flink (True Streaming):     Spark (Micro-batching):
  |||||  → Process          |||||  → Batch 1 → Process
  Time →                     |||||  → Batch 2 → Process
                             Time →
```

12.5 Performance Benchmarks (Karimov et al., ICDE 2018)

Windowed Aggregation Test:

- **Storm:** Good scaling, but latency spikes at higher throughput
- **Spark:** Good scaling, slight offset due to batching
- **Flink:** Best latency characteristics, good scaling

Key Findings:

- Good scaling with Spark & Flink
- Flink shows lowest and most consistent latency
- Spark has slight offset due to micro-batch overhead
- Storm struggles with maximum throughput scenarios

PART VI: HPC AND BIG DATA CONVERGENCE

13. HPC vs. Big Data: Fundamental Differences

13.1 Scheduling Differences

Aspect	HPC	Big Data (Hadoop v1)
Resource Control	Fine-grained (cores, accelerators, memory, time)	Integrated scheduler (Job Tracker)

Table 11 – continued

Aspect	HPC	Big Data (Hadoop v1)
Scheduler Tools	Slurm, Grid Engine, Moab, LoadLeveler	Job Tracker + Task Trackers
Fail-save	External checkpointing	Job Tracker manages failure
Programming	C/C++, Fortran, Python	Java, Scala, Python
Parallelism	SIMD/MIMD	SIMD (MapReduce)

13.2 Software Stack Comparison

```
┌──────────────────────────┬──────────────────────────┐
│   DATA ANALYTICS         │   COMPUTATIONAL SCIENCE  │
│      ECOSYSTEM           │       ECOSYSTEM          │
├──────────────────────────┼──────────────────────────┤
│ Mahout, R, Applications  │ Applications, Community  │
│ Hive, Pig, Sqoop, Flume  │ Codes                    │
│ Map-Reduce, Storm        │ FORTRAN, C, C++, IDEs    │
│ Hbase (key-value)        │ Domain-specific Libs     │
│ HDFS                     │ MPI/OpenMP, Numerical    │
│                          │ Lustre, Batch Scheduler  │
├──────────────────────────┼──────────────────────────┤
│ VMs, Cloud, Kubernetes   │ Containers (Singularity) │
│ Linux OS                 │ Linux OS                 │
├──────────────────────────┼──────────────────────────┤
│ Ethernet, Local Storage  │ Infiniband, SAN, GPUs    │
│ Commodity X86            │ X86 Racks, Accelerators  │
└──────────────────────────┴──────────────────────────┘
```

13.3 Convergence Patterns

From Application/User Perspective:

- Traditionally, data and compute in close proximity (data center)
- "Data concentration" gets reduced towards edges
- Developments driven by velocity, veracity, variety
- Dealing with data "deluge" harder at edges
- Edge devices produce high data rates: microscopes, gene sequencers, sensor networks

The HPC-HDA Cycle:

```
REALITY → Observations → Induction (patterns) → Theories → Deduction (predictions) → Predictions → (back
↪  to Observations)
        ↑_____________________________HDA_____________________________↓
        ←_____________________________HPC_____________________________→
```

- **Abduction:** Making guesses, discovery of patterns and anomalies
- **Induction:** Inferring generalizations from sampling
- **Deduction:** Drawing necessary conclusions from mathematical models

---

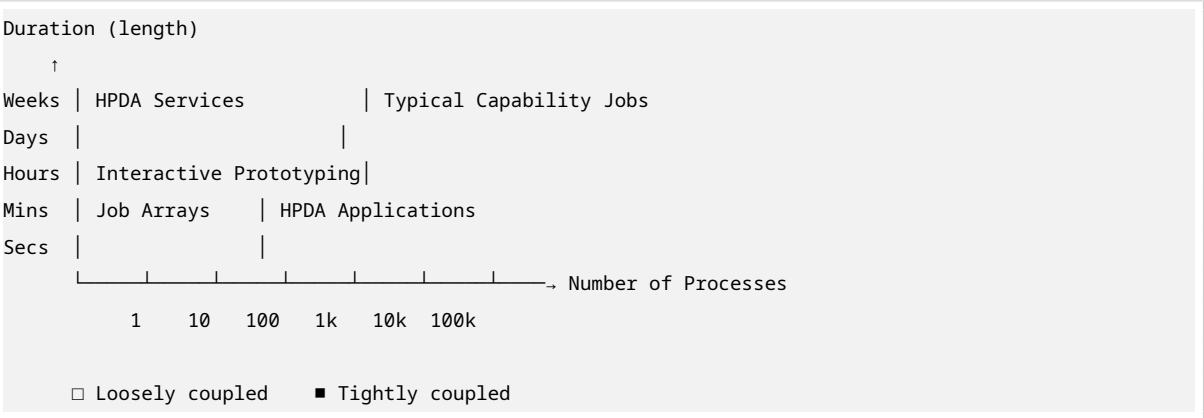
14. Scheduler Evolution and Convergence

14.1 Why Unified Scheduling?

Today's HPC Systems Face Diverse Workloads:

- HPDA (High Performance Data Analytics) services
- Capacity supercomputing jobs (simulations)
- Interactive prototyping
- Job chains (workflow dependencies)

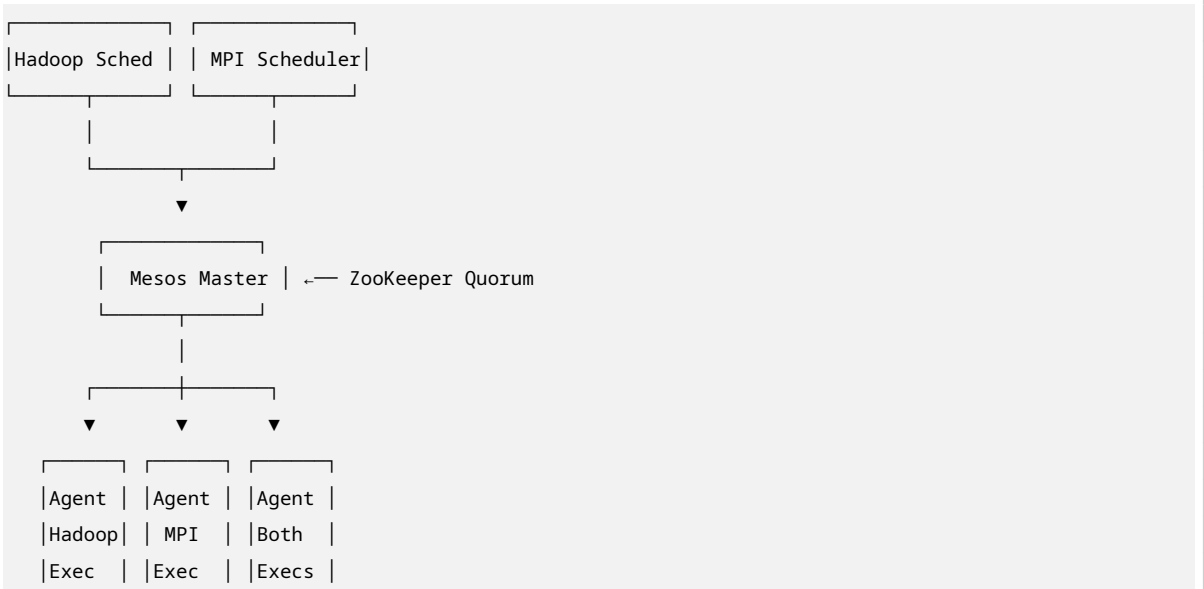
Workload Characteristics:



14.2 Apache Mesos: Unified Scheduler

Concept: Abstract scheduling out of big data framework to allow flexible hardware provisioning

Architecture:



```
└──────┘ └──────┘ └──────┘
```

14.3 Scheduler Performance Comparison

Overhead Analysis ($\Delta T = T\_total - T\_job$):

Scheduler	Overhead Scaling	Characteristics
Slurm	$\Delta T = 2.2 \cdot n^{1.3}$	Good for HPC, moderate overhead
Grid Engine	$\Delta T = 2.8 \cdot n^{1.3}$	Similar to Slurm
Mesos	$\Delta T = 3.4 \cdot n^{1.1}$	Better scaling for many small tasks
Hadoop YARN	$\Delta T = 33 \cdot n^{1}$	Higher overhead, but designed for big data

Key Insight: Modern schedulers up to factor 10 faster in task scheduling overhead. Important for many small jobs or long process chains in iterative analysis.

14.4 Big Data on HPC: Collocation Research

Challenge: HPC systems have "holes" between large parallel jobs

Opportunity: Fill these holes with small, time-independent analytics jobs

Research Question: How to efficiently mix HPC and Big Data workloads?

Visualization of Workload Patterns:

```
Big Data Workload:    Many small, short jobs (fragmented)
HPC Workload:         Few large, long jobs (blocks)
Mixed Workload:       Combination with potential for better utilization
```

PART VII: COMPLETE QUESTION BANK

Section A: Basic Understanding (Definition & Concept)

Q1: What is distributed computing?

A: A computer program that runs within a distributed system, where multiple autonomous computers work together to appear as a single coherent system to the user.

Q2: Define the 5 Vs of Big Data.

A:

- **Volume:** Amount of data (TB to EB)
- **Velocity:** Speed of data generation and processing
- **Variety:** Different types and sources of data
- **Veracity:** Quality and trustworthiness of data
- **Value:** Extractable insights and benefits

Q3: What is the difference between vertical and horizontal scaling?

A: Vertical scaling adds resources to a single machine (scale-up), while horizontal scaling adds more machines to the system (scale-out).

Q4: What is HDFS?

A: Hadoop Distributed File System - a distributed, scalable, and portable file system written in Java that stores data across multiple commodity servers.

Q5: Define MapReduce.

A: A programming model for processing large datasets in parallel across a distributed cluster, consisting of Map (filter/sort) and Reduce (aggregate) phases.

Q6: What is an RDD?

A: Resilient Distributed Dataset - Spark's fundamental data structure that is fault-tolerant, distributed across nodes, and supports parallel operations.

Q7: What is the difference between transformations and actions in Spark?

A: Transformations are lazy operations that create new RDDs (map, filter). Actions trigger computation and return values (collect, count, save).

Q8: Define stream processing.

A: Processing of continuous, unbounded data streams in real-time or near real-time, as opposed to batch processing of fixed datasets.

Q9: What is Apache Spark?

A: A fast, general-purpose, in-memory distributed computing engine for large-scale data processing with APIs in Scala, Java, Python, and R.

Q10: What is Apache Flink?

A: A stream processing framework that supports true streaming with low latency, high throughput, and exactly-once processing semantics.

Section B: Intermediate Understanding (How & Why)

Q11: Why is horizontal scaling preferred for Big Data?

A:

- Cost-effective commodity hardware
- Fault tolerance (no single point of failure)
- Virtually unlimited scalability
- Can distribute computing to data (data locality)
- Handles node failures gracefully

Q12: How does HDFS achieve fault tolerance?

A: Through block replication (default factor of 3). Each data block is stored on multiple DataNodes. If one fails, data is available from replicas. NameNode tracks block locations.

Q13: Why is MapReduce criticized by the database community?

A: Missing core DBMS features: indexing (only brute force), updates, transactions, integrity constraints, and views. It's a step backwards for data-intensive applications.

Q14: How does Spark achieve better performance than Hadoop MapReduce?

A: Spark uses in-memory computing, keeping data in RAM between operations. Hadoop writes intermediate results to disk after each MapReduce job. Spark's DAG execution engine optimizes workflows.

Q15: Why is lazy evaluation important in Spark?

A: It allows the optimizer to build a complete execution plan (DAG) before running, enabling optimizations like pipelining, predicate pushdown, and avoiding unnecessary computations.

Q16: How does Spark handle fault tolerance without replication?

A: Through RDD lineage. If a partition is lost, Spark can recompute it from the parent RDDs using the recorded transformation history, rather than relying on data replication.

Q17: Why is stream processing essential for autonomous driving?

A: Autonomous driving requires real-time sensor fusion and decision making. Batch processing introduces unacceptable latency. Stream processing handles continuous data with millisecond latency.

Q18: How does Spark Streaming differ from Flink in processing model?

A: Spark Streaming uses micro-batching (discretized streams), collecting data in small batches. Flink uses true streaming, processing each event individually as it arrives.

Q19: Why is scheduler evolution important for HPC-Big Data convergence?

A: Modern systems face diverse workloads (HPDA services, simulations, interactive jobs). Unified schedulers like Mesos can efficiently allocate resources across different job types, improving cluster utilization.

Q20: How does the convergence of HPC and Big Data benefit scientific research?

A: Enables the full data lifecycle: observations → induction (pattern discovery) → theories → deduction (simulations) → predictions → validation. Supports digital twins and physics-informed ML.

Section C: Advanced & Research Level (Analysis & Synthesis)

Q21: Analyze the trade-offs between true streaming (Flink) and micro-batching (Spark Streaming).

A:

Aspect	True Streaming (Flink)	Micro-batching (Spark)
Latency	Lower (event-by-event)	Higher (batch overhead)
Throughput	Comparable at scale	Slightly higher for some workloads
Fault Tolerance	Checkpointing overhead per event	Checkpointing per batch
State Management	Fine-grained, continuous	Batch-granular
Windowing	Flexible (event time, processing time)	Primarily processing time
Use Case	Fraud detection, real-time alerts	Log processing, analytics

Research Insight: The choice depends on latency requirements and the cost of state management. Flink's approach is more theoretically sound for streaming semantics, while Spark's can be more efficient for certain throughput-bound workloads.

Q22: Evaluate the "Shared Memory Map Communicates" model for in-memory processing.

A: This model (shown in lecture slides) represents in-memory processing where maps communicate through shared memory rather than disk or network shuffle.

Advantages:

- Eliminates disk I/O bottleneck
- Enables iterative algorithms (ML, graph)
- Supports complex analytics chains
- Low latency for intermediate results

Challenges:

- Memory capacity limits
- Garbage collection overhead (JVM)
- Cache coherence in distributed setting
- Fault recovery complexity

Research Direction: Tungsten execution engine in Spark addresses JVM overhead through off-heap memory management and code generation.

Q23: Compare and contrast the Data Analytics Ecosystem vs. Computational Science Ecosystem.

A:

Layer	Data Analytics	Computational Science
Applications	Mahout, R, Business Apps	Domain-specific codes
Languages	Java, Scala, Python, SQL	Fortran, C, C++
Middleware	MapReduce, Hive, HBase	MPI, OpenMP, Numerical Libs
Storage	HDFS (commodity)	Lustre, SAN (high-performance)
Network	Ethernet	Infiniband

Table 14 – continued

Layer	Data Analytics	Computational Science
Hardware	Commodity X86	X86 + GPUs/Accelerators

Convergence Trends:

- Both moving to Linux containers
- Big Data adopting HPC interconnects
- HPC systems adding Big Data frameworks
- Unified schedulers (Mesos, Slurm with Big Data support)

Q24: Analyze the evolution from RDD → DataFrame → Dataset in Spark.

A:

RDD (2011):

- Pros: Type-safe, functional API, flexible
- Cons: No optimization, JVM overhead, manual performance tuning

DataFrame (2013):

- Pros: Catalyst optimizer, efficient binary representation, SQL compatibility
- Cons: Runtime type checking (not compile-time safe), less flexible UDFs

Dataset (2015):

- Pros: Best of both worlds - compile-time type safety + Catalyst optimization
- Cons: Only available in Scala/Java (not Python)

Research Significance: This evolution reflects the tension between expressiveness and performance in distributed systems. The Tungsten engine further addresses JVM limitations.

Q25: Design a system for real-time traffic monitoring using stream processing.

A:

Requirements:

- Low latency (< 1 second for alerts)
- High throughput (thousands of events/second)
- Fault tolerance
- Integration with historical data

Architecture:

```
Data Sources: Toll stations, GPS, mobile apps, weather sensors
                    ↓
            Apache Kafka (message bus)
                    ↓
            Apache Flink (stream processing)
```

```
          - Windowing: 1-minute tumbling windows
          - State: Current traffic density per segment
          - Alerts: Anomaly detection for congestion
                  ↓
          Outputs: Dashboards, traffic signals, navigation apps
                  ↓
          HDFS/HBase (historical storage for ML model training)
```

Key Design Decisions:

- Flink for true streaming (sub-second latency)
- Kafka for durability and replay capability
- Windowed aggregations for trend detection
- Separate hot path (alerts) and cold path (analytics)

Q26: Discuss the challenges of running Big Data frameworks on HPC systems.

A:

Technical Challenges:

1. **Storage Mismatch:** HDFS vs. parallel file systems (Lustre)
2. **Network Topology:** Ethernet vs. Infiniband
3. **Scheduling Conflicts:** Batch HPC jobs vs. service-oriented Big Data
4. **Software Stacks:** Different libraries, dependencies

Research Solutions:

- **Apache Mesos:** Unified resource allocation
- **Spark on HPC:** Using Lustre instead of HDFS
- **Containerization:** Singularity for HPC-compatible containers
- **Burst Buffers:** SSD-based intermediate storage

Opportunity: HPC systems have idle resources between large jobs. Big Data analytics can fill these gaps, improving overall utilization.

Q27: How does the "Computing Continuum" concept extend cloud computing?

A: The Computing Continuum (2018+) integrates:

- **Edge Computing:** Processing near data sources (low latency)
- **Fog Computing:** Intermediate layer between edge and cloud
- **Cloud Computing:** Centralized, scalable resources
- **HPC Centers:** High-performance specialized computing

Benefits:

- Reduced bandwidth consumption
- Lower latency for time-critical applications

- Privacy (sensitive data processed locally)
- Fault tolerance (distributed processing)

Challenges:

- Programming complexity across layers
- Data consistency and synchronization
- Resource management across heterogeneous infrastructure
- Security across distributed boundaries

Q28: Evaluate the claim "Big Data frameworks are not just data-intensive applications but service-oriented frameworks."

A:

Evidence Supporting:

- Long-running services (HiveServer2, Spark Thrift Server)
- Interactive querying (not just batch jobs)
- Multi-tenancy and concurrent users
- Resource negotiation (YARN, Mesos)
- Dynamic scaling

Implications for Scheduling:

- Cannot treat as simple MPI jobs
- Need for reservation-based scheduling
- Service level agreements (SLAs)
- Mix of interactive and batch workloads

Research Direction: Kubernetes-based orchestration, serverless big data (AWS Lambda + Spark), and auto-scaling policies.

Q29: Analyze the performance characteristics of Storm, Spark, and Flink for windowed aggregations.

A: (Based on Karimov et al. ICDE 2018)

Methodology: Windowed aggregation with varying throughput (max vs. 90%) and cluster sizes (2, 4, 8 nodes)

Results:

- **Storm:** Good scaling but latency increases significantly at max throughput. Struggles with back-pressure.
- **Spark:** Consistent latency, slight offset due to micro-batch overhead. Good scaling with cluster size.
- **Flink:** Lowest and most stable latency. Best scaling characteristics. Efficient state management.

Key Insights:

- At 90% throughput (not max), all systems show better stability
- Flink's true streaming model pays off for latency-sensitive applications
- Spark's throughput can be higher for some batch-like workloads
- Storm requires careful tuning for production use

Q30: Design a research proposal for "Physics-Informed Neural Networks on Distributed Systems."

A:

Problem: Traditional ML ignores physical constraints. Scientific simulations need to incorporate domain knowledge.

Approach:

1. **Distributed Training:** Use Spark/Flink for parallel training across cluster
2. **Physics Constraints:** Embed differential equations as loss function terms
3. **Data Fusion:** Combine simulation data (HPC) with observational data (Big Data)

System Architecture:

```
HPC Simulation → HDFS/Parallel FS → Spark MLlib
                                        ↓
Observational Data → Kafka → Flink Stream Processing
                                        ↓
                                Physics-Informed Loss Function
                                        ↓
                                Distributed Training (Horovod/TensorFlowOnSpark)
                                        ↓
                                Digital Twin Model
```

Research Questions:

- How to efficiently compute physics constraints in distributed setting?
- Communication patterns for coupled physics-ML training
- Fault tolerance for long-running scientific ML training

Section D: Industrial & Practical Questions

Q31: When should a company choose Hadoop over Spark?

A:

- **Choose Hadoop when:**
- Budget constraints (cheaper hardware, mature ecosystem)
- Primarily batch processing with simple workflows
- Need for stable, proven technology

- Existing Hadoop infrastructure
- Very large datasets that don't fit in memory
- **Choose Spark when:**
  - Need for iterative algorithms (ML, graph)
  - Interactive analytics required
  - Real-time stream processing
  - Complex multi-stage pipelines
  - In-memory caching beneficial

Q32: How to migrate from Hadoop MapReduce to Spark?

A:

1. **Assessment:** Identify MapReduce jobs, data flows, dependencies
2. **Infrastructure:** Ensure sufficient RAM for in-memory processing
3. **Code Migration:**

- Map → map/flatMap
- Reduce → reduceByKey/aggregate
- Custom InputFormats → Spark Hadoop API

4. **Optimization:**

- Use DataFrames/Datasets instead of RDDs where possible
- Optimize partition count
- Cache intermediate results strategically

5. **Testing:** Validate results match original MapReduce output
6. **Monitoring:** Use Spark UI for performance tuning

Q33: What are the cost considerations for cloud-based Big Data?

A:

- **Compute:** On-demand vs. reserved instances, spot instances for batch
- **Storage:** HDFS on EBS vs. S3, data transfer costs
- **Network:** Inter-zone traffic, egress costs
- **Services:** Managed services (EMR, Dataproc) vs. self-managed
- **Hidden costs:** Data ingress, API calls, long-running clusters

Q34: How to handle data skew in Spark?

A:

- **Symptoms:** Some tasks take much longer, uneven executor utilization
- **Solutions:**
- Salting: Add random prefix to keys for aggregation

- Adaptive Query Execution (AQE): Dynamically optimize skew joins
- Custom partitioner: Distribute skewed keys evenly
- Two-stage aggregation: Pre-aggregate locally

Q35: Design a Lambda architecture vs. Kappa architecture decision framework.

A:

Lambda Architecture:

```
Speed Layer (Stream) ──┐
                       ├──→ Serving Layer → Queries
Batch Layer (Historical)─┘
```

- **Pros:** Fault tolerance through recomputation, handles late data
- **Cons:** Code duplication (batch + stream logic), complexity

Kappa Architecture:

```
Stream Processing Only → Serving Layer → Queries
```

- **Pros:** Single codebase, simpler
- **Cons:** Requires replay capability, harder for complex historical analysis

Decision Framework:

- Use Lambda when: Complex historical analytics, regulatory requirements for recomputation
- Use Kappa when: Real-time is primary, simple replay available, stream processing mature

---

PART VIII: KEYWORDS & GLOSSARY

A

- **Action (Spark):** Operation that triggers computation and returns result
- **Apache Flink:** True streaming processing framework
- **Apache Hive:** Data warehouse infrastructure on Hadoop
- **Apache Mesos:** Cluster manager for mixed workloads
- **Apache Spark:** In-memory distributed computing engine
- **Apache Storm:** Early stream processing framework
- **Apache YARN:** Resource manager for Hadoop 2.0+

B

- **Batch Processing:** Processing fixed, bounded datasets
- **Block (HDFS):** Basic unit of storage (default 128MB)
- **Broadcast Variable:** Read-only variable cached on each worker
- **Big Data:** Datasets too large/complex for traditional tools

C

- **Caching:** Storing frequently accessed data in fast memory
- **Catalyst Optimizer:** Spark SQL query optimizer
- **Checkpoint:** Saving RDD/stream state for fault recovery
- **Cloud Computing:** On-demand computing resources over network
- **Cluster:** Group of connected computers working together
- **Column Store (HBase):** Database storing data by columns

D

- **DAG (Directed Acyclic Graph):** Spark's execution plan representation
- **DataFrame:** Spark distributed collection with schema
- **Data Locality:** Moving computation to data location
- **DataNode (HDFS):** Worker node storing actual data blocks
- **Dataset:** Type-safe DataFrame (Scala/Java)
- **DStream:** Spark Streaming abstraction
- **Distributed Computing:** Computation across multiple machines

E

- **Edge Computing:** Processing near data sources
- **Executor:** Spark worker process running tasks
- **Exactly-once Semantics:** Guarantee each event processed once

F

- **Fault Tolerance:** System continues operating despite failures
- **Flink:** Apache stream processing framework

G

- **Garbage Collection:** JVM memory management (challenge for Big Data)
- **GraphX:** Spark graph processing library

H

- **Hadoop:** Open-source distributed computing framework
- **HDFS:** Hadoop Distributed File System
- **Horizontal Scaling:** Adding more machines (scale-out)
- **HiveQL:** SQL-like query language for Hive
- **HPC:** High Performance Computing

I

- **In-Memory Computing:** Processing data in RAM, not disk
- **IoT:** Internet of Things
- **Iterative Algorithm:** Repeated execution until convergence (ML)

J

- **Job Tracker (Hadoop 1):** Master coordinating MapReduce jobs
- **JVM:** Java Virtual Machine (Spark runs on JVM)

K

- **Kafka:** Distributed streaming platform
- **Key-Value Store:** NoSQL database (HBase)

L

- **Lambda Architecture:** Batch + speed layer design
- **Lazy Evaluation:** Delay computation until necessary
- **Lineage:** RDD transformation history for fault recovery

M

- **Map Phase:** Extract and transform input data
- **MapReduce:** Distributed programming model
- **Mesos:** Cluster resource manager
- **Micro-batching:** Processing streams in small batches
- **MLlib:** Spark machine learning library

N

- **NameNode (HDFS):** Master managing file system metadata
- **NoSQL:** Non-relational database systems

P

- **Partition:** Subset of distributed data
- **Prefetching:** Predicting and loading data before needed

R

- **RDD:** Resilient Distributed Dataset
- **Reduce Phase:** Aggregate mapped data
- **Replication:** Copying data for fault tolerance
- **Resource Manager:** Allocates cluster resources (YARN, Mesos)

S

- **Scale-Out:** Horizontal scaling
- **Scale-Up:** Vertical scaling
- **Shared Nothing:** Independent nodes with local storage
- **Shuffle:** Data redistribution between stages
- **Spark Core:** Foundation engine for Spark
- **Spark SQL:** Spark module for structured data
- **Spark Streaming:** Spark micro-batch stream processing
- **State:** Maintained information in stream processing

- **Stream Processing:** Continuous data processing
- **Structured Streaming:** Spark's unified stream/batch API

T

- **Task:** Unit of work sent to executor
- **Task Tracker (Hadoop 1):** Worker node task manager
- **Transformation:** Lazy RDD operation creating new RDD
- **Tungsten:** Spark's optimized execution engine
- **Tuple:** Ordered collection of elements (key-value pair)

U

- **UDF:** User Defined Function
- **Unified Analytics:** Combining batch, streaming, ML, SQL

V

- **Velocity:** Speed of data generation/processing
- **Veracity:** Data quality and trustworthiness
- **Vertical Scaling:** Adding resources to single machine
- **Volume:** Amount of data

W

- **Window:** Time-based or count-based data grouping
- **Worker Node:** Cluster node executing tasks
- **Workflow:** Sequence of dependent jobs (Oozie)

Y

- **YARN:** Yet Another Resource Negotiator (Hadoop 2.0+)

Z

- **ZooKeeper:** Coordination service for distributed systems

---

PART IX: EXAM PREPARATION CHECKLIST

Core Concepts to Master

- Distributed systems definition and characteristics
- 5 Vs of Big Data with examples
- Vertical vs. Horizontal scaling trade-offs
- HDFS architecture (NameNode, DataNode, blocks, replication)
- MapReduce phases (Map, Shuffle, Reduce) with word count example
- Hadoop 1.0 ecosystem components and their roles
- MapReduce limitations and criticism from DB perspective

- In-memory processing advantages and techniques
- Spark architecture (Core, SQL, Streaming, MLlib, GraphX)
- RDD properties, operations, lineage, fault tolerance
- DataFrame vs. Dataset vs. RDD evolution
- Spark vs. Hadoop performance characteristics
- Stream processing concepts and models
- Flink vs. Spark Streaming architecture differences
- HPC vs. Big Data software stacks
- Scheduler evolution (Slurm, Mesos, YARN)
- HPC-Big Data convergence patterns

Mathematical/Analytical Skills

- Calculate HDFS storage requirements with replication
- Estimate MapReduce job completion times
- Compare latency/throughput trade-offs
- Analyze scaling behavior (strong vs. weak scaling)

Programming/Practical Skills

- Write basic MapReduce pseudocode
- Write Spark transformations and actions
- Design stream processing pipelines
- Optimize Spark jobs (caching, partitioning)

Research-Level Understanding

- Physics-informed neural networks on distributed systems
- Digital twins combining simulation and data
- Edge-to-cloud continuum architectures
- Unified scheduling for mixed workloads
- Fault tolerance in long-running ML training

PART X: IMPORTANT REFERENCES FROM LECTURES

Key Papers

1. Dean, J. & Ghemawat, S. (2004). "MapReduce: Simplified Data Processing on Large Clusters." OSDI.
2. Ghemawat, S., Gobioff, H., & Leung, S. (2003). "The Google File System." SOSP.
3. Karimov, J. et al. (2018). "Benchmarking Distributed Stream Data Processing Systems." ICDE.
4. Lindsay, D. et al. "The evolution of distributed computing systems: from fundamental to new frontiers." Computing 103.
5. Donta, P.K. et al. (2023). "Exploring the Potential of Distributed Computing Continuum Systems." Computers 12.

Key Technologies Timeline

- 1955: Mainframe
- 1962: Cluster
- 1967: Network Computing
- 1978: Home Computer
- 1994: WWW
- 1999: Grid Computing, P2P
- 2004: Mobile Computing
- 2006: Cloud Computing, Hadoop
- 2008: IoT
- 2009: Spark (Berkeley), Fog/Edge Computing
- 2010: Hive
- 2011: RDD
- 2013: DataFrame
- 2015: Dataset, Flink
- 2018: Computing Continuum

Document prepared for: TU Dresden M.Sc. Computer Science — Distributed Computing (Lectures 10 & 11) **Coverage:** Fundamentals to Research Level **Last Updated:** 2026